DiffKV

Anchor + Low-Rank Differential KV-Cache Compression for Long-Context Inference on Commodity Unified-Memory Hardware

Technical Report

Generated July 7, 2026

Om Chimurkar

https://github.com/Omc12/Differential-KV

A sparse KV-cache inference runtime that compresses each transformer KV block to an *anchor token*, a rank-32 joint SVD *delta*, and a set of *exact residual tokens*, paired with a bounded dense recency window and a fused, routed decode kernel that scores queries in low-rank space without ever decompressing the cache and attends the residuals exactly. The bounded store reaches 64k context where a dense cache exhausts an 8.6 GB Apple M3, and the residuals recover a buried passcode exactly at every context. Evaluated on Qwen2.5-1.5B (int4). All numbers in this report are measured; no value is fabricated.

Abstract

The key–value (KV) cache is the memory wall of long-context transformer inference: its size grows linearly with sequence length, and on commodity hardware it, not the model weights, is what first exhausts memory. We present DiffKV, a KV-cache compression runtime that keeps recent tokens exact and compresses older ones. Each block of B=256 tokens is reduced to an *anchor* token (kept exact), a rank-r joint $K|V$ truncated-SVD *delta* that captures the block's shared structure, and a budget of R *exact residual tokens*—the rows the low-rank basis reconstructs worst, which are precisely the distinctive tokens (digits, names, codes) that verbatim recall depends on. A bounded dense recency window holds the newest tokens uncompressed. At decode time a fused, `mx.compiled` kernel routes to the top-K relevant blocks, scores the query against them *in low-rank space without ever decompressing K*, attends the residuals and recency window exactly, and merges the two halves with a flash-style log-sum-exp reduction. Prefill uses a training-free block-sparse attention so its cost grows sub-quadratically.

We implement DiffKV entirely in MLX and evaluate it on Qwen2.5-1.5B (int4) against a dense full-KV baseline on the *same* weights, on an Apple M3 with 8.6 GB of unified memory. DiffKV holds a bounded KV state that is $1.44\times$–$2.25\times$ smaller per block than a dense cache (the residual budget R is the knob), recovers a buried passcode *exactly* at every context from 4k to 64k, and reaches 64k—where the dense full-KV baseline runs out of memory on the same host—while its sparse prefill overtakes dense at long context. The cost is decode throughput: reconstructing and merging the sparse representation makes per-token decode slower than a dense cache while the dense cache still fits. We report this trade-off in full, isolate each component with ablations, and characterize the residual budget as an explicit accuracy–memory–speed dial. All numbers in this report are measured on the described host; none are estimated. A CUDA/Triton decode path exists in the codebase but is not evaluated here, and the report reserves space for it without restructuring.

Contents

List of Figures

List of Tables

1. Introduction

Autoregressive transformer inference caches the key and value projections of every past token so that each new token attends over the whole context without recomputation. This KV cache is what makes decoding fast, but it is also what makes long context expensive: its size grows linearly with sequence length and with the number of layers, and for a small model on commodity hardware it overtakes the weights as the dominant consumer of memory. For Qwen2.5-1.5B the int4 weights occupy roughly a gigabyte, while a full fp16 KV cache costs about 28 KB per token across all layers—so a 64k-token conversation asks for close to 1.8 GB of cache on top of the model, on a machine that may have only 8 GB of unified memory shared with the operating system, the display server, and every other process. The cache, not the model, is the wall.

A large body of work attacks this wall by making the cache smaller or sparser: low-bit quantization of K/V, eviction of tokens deemed unimportant, and low-rank factorization of the cache tensors. Each trades some fidelity for memory. The failure mode that matters in practice is not average perplexity but *verbatim recall*: a summary that paraphrases the document is often acceptable, but a summary that reports the wrong passcode, account number, or name is not. Aggressive compression tends to destroy exactly the low-frequency, high-information tokens on which such recall depends, because those tokens are outliers that a shared low-rank or low-bit representation reconstructs worst.

DiffKV is a KV-cache compression runtime built around this observation. It keeps a bounded window of recent tokens exact, and compresses each older block of B=256 tokens into three parts: an *anchor* (the block's first token, kept exact and used as the baseline for the rest), a rank-r joint $K|V$ truncated-SVD *delta* that captures the structure the block's tokens share, and a small budget of *exact residual tokens*—selected as the rows with the largest reconstruction error, which are precisely the distinctive tokens the low-rank basis fails to represent. The anchor-plus-delta captures the smooth, shared part of the block cheaply; the residuals rescue the outliers exactly. Decode never materializes the full cache: a fused kernel scores the query against each block in the rank-r subspace, attends the exact residuals and the recency window directly, and combines the two contributions with a numerically stable log-sum-exp merge. A cheap per-block router restricts the expensive part of this computation to the top-K relevant blocks, so decode cost scales with K rather than with the context length.

What this paper claims, and what it does not. We are deliberate about the trade-off, because it is easy to overstate. On our host—an Apple M3 with 8.6 GB of unified memory, running Qwen2.5-1.5B at int4—the dense full-KV cache *fits and decodes faster than* DiffKV through 32k tokens; DiffKV does not make this model decode faster where a dense cache still fits. But at 64k the dense cache exhausts the machine's memory and does not run at all, whereas DiffKV completes with the passcode intact. What DiffKV provides is therefore (i) a KV state that is bounded by a fixed block pool and is $1.44\times$–$2.25\times$ smaller per block than a dense cache, so the cache stops being the term that grows without limit and 64k becomes reachable where dense cannot; (ii) *exact* recovery of a buried passcode at every context we tested, from 4k to 64k, preserved by the residual mechanism; and (iii) a sparse prefill whose sub-quadratic cost lets long-context prefill overtake the dense baseline once the context is large enough. The price is per-token decode throughput *in the regime where a dense cache still fits*. We report the full picture—including where DiffKV loses—and we treat the residual budget R as an explicit dial between memory, accuracy, and speed.

Contributions.

- An **anchor + low-rank + exact-residual** block representation (§4) that decouples the smooth, shared part of a KV block (compressed) from its distinctive outlier tokens (kept exact), giving a tunable compression ratio while preserving verbatim recall.
- A **fused, routed decode kernel** (§6) that scores queries in the low-rank subspace without decompressing K, attends residuals and recency exactly, and merges the halves with a flash-style reduction—so decode cost scales with the routed block count K, not the context length.
- A **bounded-memory runtime** (§5) with streaming prefill compression, a fixed block pool, a prefill-to-decode memory release, and a training-free block-sparse prefill, implemented entirely in MLX for Apple unified memory.
- A **measured, self-consistent evaluation** (§8) against a dense baseline on identical weights, reporting footprint, prefill, decode, and exact recall from 4k to 64k, with component ablations and a residual-budget sweep. We state the decode-throughput cost plainly rather than hiding it.

The rest of the paper follows the system's own execution order: background and the memory arithmetic that motivates the design (§2), a component overview (§3), the compression pipeline (§4), the runtime and memory architecture (§5), the fused decode kernel (§6), implementation notes (§7), evaluation (§8), analysis (§9), and limitations (§10).

2. Background and Motivation

2.1. The KV-cache memory arithmetic

Consider a decoder-only transformer [14] with L layers, H_{kv} key/value heads (grouped-query attention [1] shares each KV head across several query heads), and head dimension d. Caching one token's keys *and* values across all layers, in fp16, costs

$$b_{\text{tok}} \;=\; 2 \cdot L \cdot H_{\text{kv}} \cdot d \cdot 2 \text{ bytes}. \tag{1}$$

For our evaluation model (Qwen2.5-1.5B: L=28, H_{kv}=2, d=128) this is 28,672 bytes per token—about 28 KB. The cache for a sequence of length N is therefore $N \cdot b_{\text{tok}}$ and grows without bound in N: 16k tokens cost 0.47 GB, 32k cost 0.94 GB, and 64k cost 1.8 GB. Against 8.6 GB of unified memory shared with the weights ($\approx$ 1 GB at int4), the OS, and transient prefill activations, the cache is the term that decides whether a long context fits.

2.2. The design space

Three broad families shrink the cache. *Quantization* stores K/V at lower precision (e.g. int4/int8, per-channel or per-token scales) [11]; it is simple and preserves every token, but uniformly low precision erodes the outlier tokens most. *Eviction/selection* keeps only a subset of tokens—attention sinks [15], heavy hitters [16], persistence-of-importance [10], prompt-aware selection [9], or query-aware block retrieval [13]; it can be very compact but risks discarding a token that a later query needed. *Low-rank factorization* exploits redundancy across tokens or channels, replacing the cache with a small basis [3, 4]; it compresses the shared structure well but, by construction, discards the low-energy directions in which distinctive tokens live. Orthogonally, KV *memory management* such as paged attention [8] reduces fragmentation without shrinking the logical cache.

The tension common to all three is that the tokens most expensive to keep—rare, high-entropy tokens such as identifiers, digits, and names—are exactly the tokens that verbatim recall needs, and exactly the tokens a shared low-rank or low-bit code represents worst. A method that only compresses the shared structure will paraphrase fluently and misquote the one fact that mattered.

2.3. Design principle: separate the smooth part from the outliers

DiffKV responds to that tension directly by splitting each block into two representations with complementary strengths. The bulk of a block—the smooth, correlated variation shared by its tokens—is captured by an anchor and a rank-r joint $K \mid V$ delta, which is cheap and reconstructs the shared structure at low error. The residual outliers—the handful of rows the low-rank basis reconstructs worst—are kept *exact*. Because the residuals are chosen by measured reconstruction error rather than by a hand-picked token class, the mechanism is content- and model-agnostic: it keeps whatever a given block's distinctive tokens happen to be. The recent past is not compressed at all; it lives in a small exact recency window, so short-range attention is always faithful. §8 shows that this split is what lets a buried passcode survive compression exactly (E5) while the low-rank part keeps the per-block budget small.

2.4. Unified memory and MLX

Our target is Apple silicon, where the CPU and GPU share one physical memory pool. This changes the accounting in two ways that shape the implementation. First, there is no host–device copy: a tensor produced on the GPU is visible to NumPy without a transfer, so the randomized SVD used during compression can run in NumPy on the CPU while the attention kernels run on the GPU, with no marshalling cost. Second, "freeing" memory means returning it to a single shared pool that the rest of the system is contending for, so bounding *peak* allocation matters as much as bounding steady state. DiffKV is built on MLX: it monkey-patches the attention of an `mlx_lm` model, keeps its compressed store in MLX fp16 arrays, compiles the decode kernel with `mx.compile`, and explicitly releases the prefill cache at the prefill→decode boundary (§5).

3. System Overview

DiffKV is structured as a thin serving stack over an MLX inference core whose attention has been replaced (Figure 1). We walk the components in the order data flows through them.

Serving. An OpenAI-compatible gateway and a continuous-batching decode engine accept requests; a session manager owns per-conversation lifecycle and residency. These layers are standard and are not the subject of this paper; they exist so that the compression runtime is exercised under a realistic request path.

Wrapper and model. `MLXDiffKVWrapper` loads an `mlx_lm` model, patches its attention, and exposes `generate()`, which tokenizes the prompt, runs prefill in fixed-size chunks, and then decodes one token at a time. `MLXQwenModel` wraps the patched model, holds a native MLX KV cache used only during prefill, and—critically—*releases* that prefill cache at the prefill→decode boundary so that decode-time memory reflects the compressed store rather than a retained full-context cache (§5).

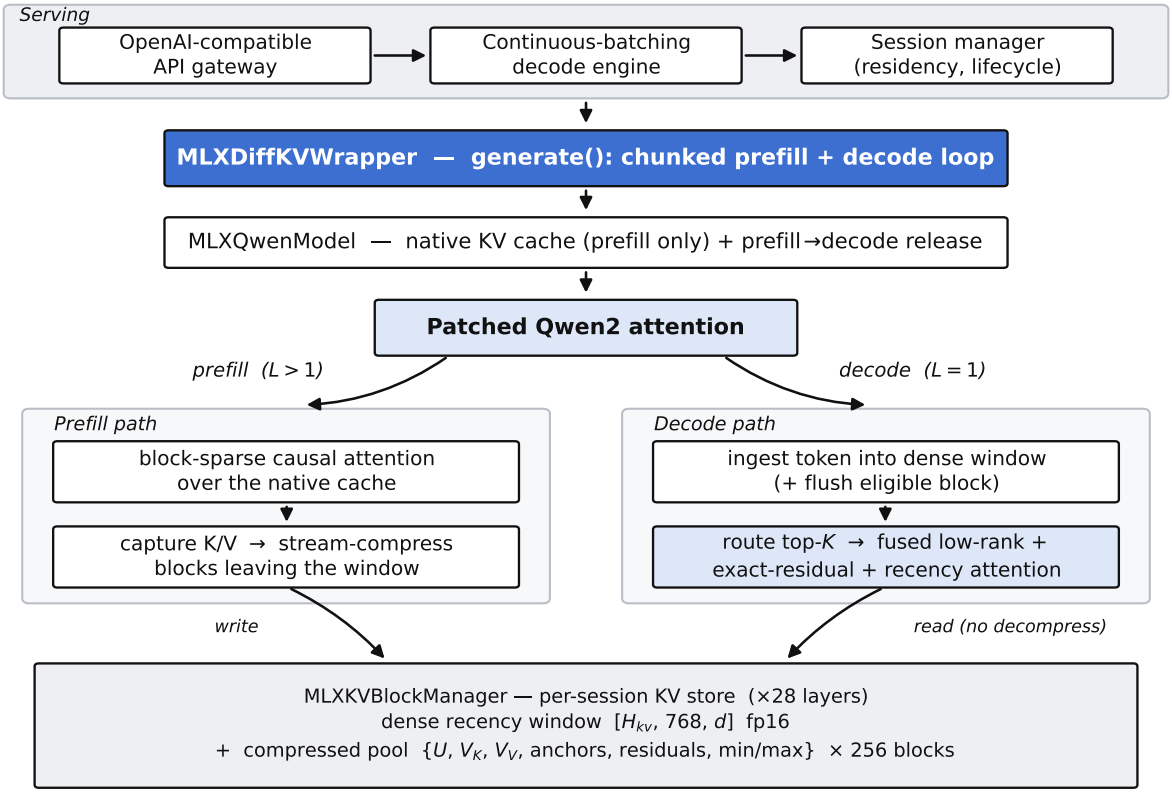

Figure 1 | DiffKV active-runtime architecture. A serving stack (gateway, batching engine, session manager) drives `MLXDiffKVWrapper`, which runs chunked prefill and a greedy or sampled decode loop over `MLXQwenModel`. The model's attention layer is monkey-patched: on prefill (L>1) it runs (block-sparse) causal attention over the native cache and captures K/V into the store; on decode (L=1) it ingests the current token and runs the fused routed sparse attention. All state lives in the per-session `MLXKVBlockManager`: a dense recency window plus a bounded compressed block pool.

Patched attention. The heart of the system is a replacement for the model's attention `__call__`. It branches on sequence length. During prefill (L>1) it computes causal attention—optionally block-sparse (§5)—over the growing native cache to produce correct hidden states, and captures each chunk's K/V into the store. During decode (L=1) it ingests the newly generated token's K/V into the dense window and computes the attention output from the compressed store via the fused routed kernel (§6).

The store. `MLXKVBlockManager` holds all persistent state, per session and per layer: a *dense recency window* of the most recent $W+B$=768 tokens in exact fp16, and a *compressed block pool* of up to M=256 blocks, each holding the anchor, low-rank factors, exact residuals, and router statistics for one 256-token span. The pool is pre-allocated and fixed in size, which is what bounds the runtime's memory (§5, Figure 3).

Two data paths, one store. Prefill *fills* the store (capture, then stream-compress the oldest blocks as the window overflows); decode *reads* it (route, reconstruct in low-rank space, attend residuals and window). The compression pipeline (§4) defines the store's contents; the runtime (§5) defines

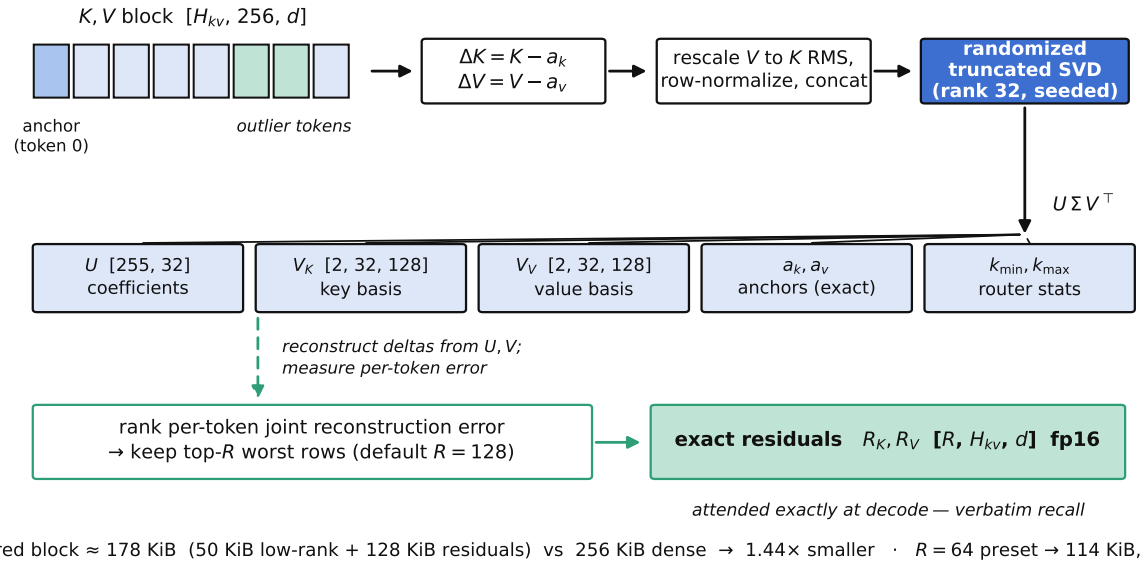

Figure 2 | Compression of one 256-token block. The anchor (token 0) is subtracted to form deltas; V is rescaled to K's RMS and the rows are L_2-normalized before a seeded randomized truncated SVD produces U, V_K, V_V. In parallel, per-token joint reconstruction error selects the top-R rows, kept as *exact* residuals R_K, R_V. Key min/max are retained for the decode router. At the default R=128 a block stores ≈ 177.9 KiB vs. 256 KiB dense (1.44×); the R=64 preset stores ≈ 113.9 KiB (2.25×).

its lifecycle; the decode kernel (§6) defines how it is read. We take these in turn.

Scope. This report documents the *active runtime*—the MLX implementation. An independent native (C++/GGML) engine and a CUDA/Triton decode kernel exist in the repository and are out of scope here; we neither evaluate them nor compare against them, and where the design ports cleanly we say so and reserve space for future measurement (§10). A separate factual-store / semantic-retrieval subsystem is present but disabled by default and is not part of the evaluated system.

4. Differential KV Compression

A block is B=256 consecutive tokens' keys and values, $K, V \in \mathbb{R}^{H_{\text{kv}} \times B \times d}$. Compression (Figure 2) runs when a block leaves the recency window. It produces four things: an anchor, a low-rank delta basis, a set of exact residual tokens, and per-block router statistics.

4.1. Anchor and delta

Token 0 of the block is the *anchor* $a_k, a_v \in \mathbb{R}^{H_{\text{kv}} \times d}$, stored exactly. Every other token is represented relative to it:

$$\Delta K_i = K_i - a_k, \qquad \Delta V_i = V_i - a_v, \qquad i = 1, \ldots, B-1. \tag{2}$$

Subtracting a within-block baseline concentrates the energy that the low-rank basis must capture, so a small rank suffices for the shared structure.

4.2. Joint $K|V$ low-rank factorization

K and V are compressed *jointly*: for each token we stack $[\Delta K_i \,\|\, \Delta V_i]$ and factor the $(B-1) \times 2H_{\text{kv}}d$ delta matrix once, so a single set of coefficients U drives both reconstructions. Two normalizations make the joint factorization well-posed. In Qwen2.5 the key norms dominate the value norms, so before factoring we rescale V's deltas to K's RMS energy (a per-block gain g, undone on the stored V_V); without this the SVD spends its budget on K and the V reconstruction is poor. We then L_2-normalize each token's stacked delta row so the factorization is not dominated by a few high-norm tokens, folding the norms back into U afterwards.

The factorization is a *randomized truncated SVD* [5] with a fixed seed (default 1234): a Gaussian range finder with power iterations, a QR step, and a small dense SVD of the projected matrix, all in fp32 in NumPy (§7 explains why NumPy). The rank is adaptive: it keeps the smallest number of components (at least 4, at most r) that explain 99.9% of the delta energy. The result, cast to fp16, is

$$U \in \mathbb{R}^{(B-1)\times r}, \quad V_K, V_V \in \mathbb{R}^{H_{\text{kv}}\times r\times d}, \tag{3}$$

so that the implicit reconstruction of any non-anchor token is

$$\hat{K}_i = a_k + s\,(U_i V_K), \qquad \hat{V}_i = a_v + s\,(U_i V_V), \tag{4}$$

with s the stored per-block scale. Crucially, decode never forms $\hat{K}_i$ explicitly (§6); Eq. (4) is the semantics, not the runtime path.

4.3. Exact residuals: rescuing the outliers

The low-rank basis reconstructs the shared structure well but, by construction, mis-reconstructs low-energy outliers—which are the distinctive tokens recall depends on. DiffKV measures this directly. For every token it computes the joint reconstruction error $e_i = \left\|\Delta K_i - \widehat{\Delta K}_i\right\|^2 + g^2\left\|\Delta V_i - \widehat{\Delta V}_i\right\|^2$ (the V term reweighted by the same gain g used above), and keeps the R highest-error tokens as *exact* fp16 residuals $R_K, R_V \in \mathbb{R}^{R\times H_{\text{kv}}\times d}$. Their low-rank twins are later excluded from the reconstruction pool so each such token is represented once, exactly (§6). Selecting by measured error, rather than by a token-type heuristic, is what makes the residual mechanism content- and model-agnostic. We validate empirically (§8, E5–E6) that (i) residuals are what make a buried passcode survive compression exactly, and (ii) the residual reconstruction K must be stored exactly—using a low-rank-reconstructed key for a residual token was measured to break recall.

4.4. Router statistics

Two cheap summaries per block support decode-time routing (§6): the element-wise key min and max over the block, $k_{\min}, k_{\max} \in \mathbb{R}^{H_{\text{kv}}\times d}$, which bound the best possible query–key score for a Quest-style relevance test.

4.5. Byte budget and complexity

Table 1 accounts for one block exactly as it is laid out in the store. The low-rank core (coefficients, both bases, anchors, min/max, scalars) is a fixed ≈ 49.9 KiB, independent of R; the residuals add $R \cdot H_{\text{kv}} \cdot d \cdot 2 \cdot 2$ bytes. At the default R=128 a block is 177.9 KiB versus 256 KiB dense (1.44×); at the R=64 memory preset it is 113.9 KiB (2.25×). The residual budget therefore *is* the compression dial,

Table 1 | Per-256-token-block storage, computed from the model dimensions and the store's fp16 layout. The low-rank core is fixed; the residual budget R sets the ratio.

Component	Bytes	Note
U coefficients $[255, 16]$	16320	low-rank
V_K, V_V $[2, 16, 128]$	32768	low-rank
anchors a_k, a_v $[2, 128]$	1024	exact
key min/max $[2, 128]$	1024	router
scale, seq_len	8	scalar
Low-rank core	**51144**	(49.9 KiB)
exact residuals R=128	131072	(128.0 KiB)
exact residuals R=64	65536	(64.0 KiB)
DiffKV block (R=128)	**182216**	**177.9 KiB**
DiffKV block (R=64)	**116680**	**113.9 KiB**
Dense block $[256, 2, 128]\times 2$	262144	256.0 KiB
Compression ratio (R=128)		1.44×
Compression ratio (R=64)		2.25×

Algorithm 1 Compress one block (streaming, batched across layers in the implementation)

Require: block keys/values $K, V \in \mathbb{R}^{H_{\text{kv}} \times B \times d}$; rank r; residual budget R

$a_k, a_v \leftarrow K_0, V_0$ ▷ anchor kept exact

$\Delta K \leftarrow K_{1:} - a_k; \quad \Delta V \leftarrow V_{1:} - a_v$

$g \leftarrow \sqrt{\|\Delta K\|^2 / \|\Delta V\|^2}$ ▷ rescale V to K's RMS

$D \leftarrow \text{rownorm}\big([\, \Delta K \,\|\, g\, \Delta V \,]\big)$; keep norms in U

$U, \Sigma, V^\top \leftarrow \textsc{RandTruncSVD}(D,\ r,\ \text{seed=1234},\ 99.9\%)$

split $V^\top \to V_K, V_V$; undo g on V_V; store scale s

$e_i \leftarrow \|\Delta K_i - \widehat{\Delta K}_i\|^2 + g^2 \|\Delta V_i - \widehat{\Delta V}_i\|^2$ ▷ joint error

$\mathcal{R} \leftarrow$ indices of the R largest e_i; $R_K, R_V \leftarrow K_{\mathcal{R}}, V_{\mathcal{R}}$ ▷ exact

$k_{\min}, k_{\max} \leftarrow \min_i K_i,\ \max_i K_i$ ▷ router bounds

return $(a_k, a_v, U, V_K, V_V, s, R_K, R_V, \mathcal{R}, k_{\min}, k_{\max})$

and the residual pool dominates the block at the default setting—a fact the memory layout makes visual (§5). Compressing a block costs one randomized SVD of an $\approx 255 \times 512$ matrix (≈ 2 ms), amortized over 256 tokens; §8 shows compression is a single-digit percentage of prefill time.

5. Runtime and Memory Architecture

The store's contents are defined by §4; this section defines its *layout* and *lifecycle*—what makes the runtime's memory bounded and its long-context prefill affordable.

5.1. Session state

Each session holds, per layer (Figure 3): a dense recency window—fixed-capacity fp16 buffers $[H_{\text{kv}},\ W{+}B,\ d]$ for the newest 768 tokens—and a compressed pool of M=256 block slots, pre-allocated for U, V_K, V_V, anchors, residuals R_K, R_V, key min/max, and per-block scalars. Because

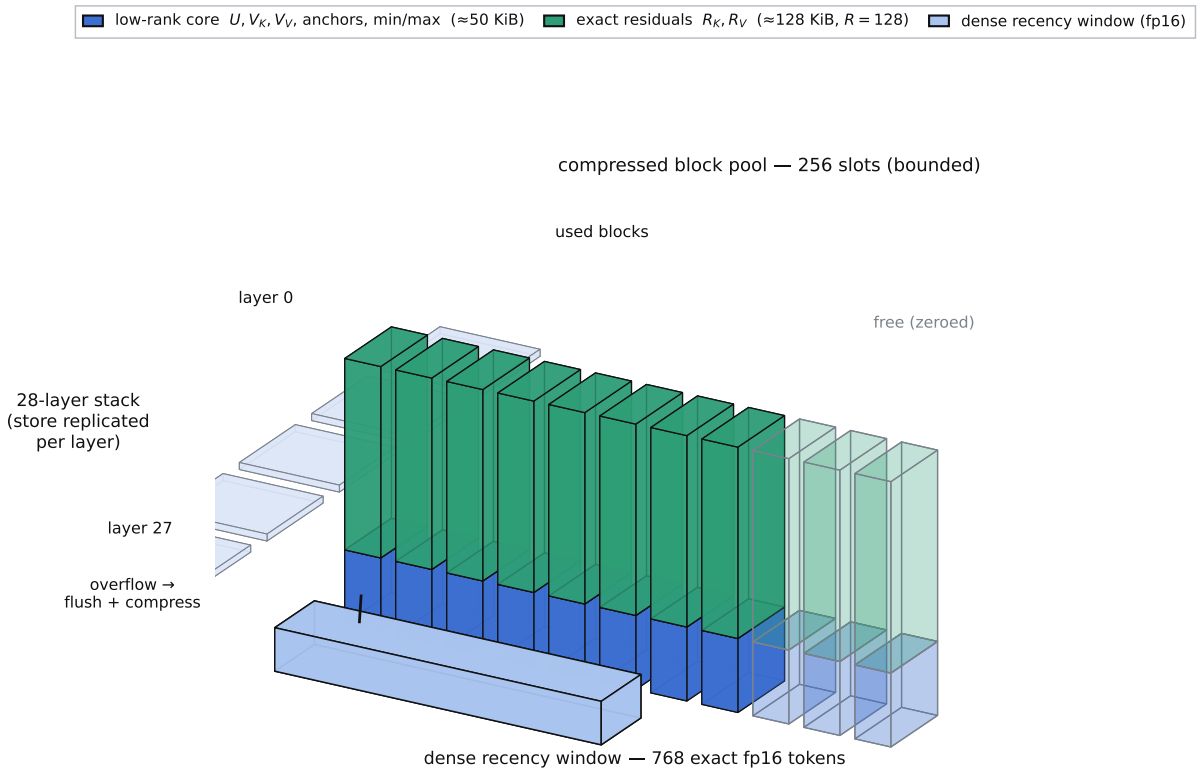

Figure 3 | The per-session KV store in three dimensions. The store is replicated across all 28 layers (left). Within a layer, the compressed pool is a bounded array of M=256 block slots; each occupied slot holds a small low-rank core and—at the default budget—a much larger block of exact residuals (residuals dominate the per-block bytes, cf. Table 1). The dense recency window (front) holds the newest 768 tokens uncompressed; when it overflows, its oldest block is flushed and compressed into a pool slot.

the pool is pre-allocated at its maximum size, the store's footprint is *bounded and context-independent* up to the pool's capacity of $256 \times 256 = 65{,}536$ compressed tokens; adding context fills empty slots rather than growing the allocation. This is the structural source of the bounded-memory behavior in §8.

5.2. Chunked, streaming prefill

Prefill consumes the prompt in fixed chunks (default 512 tokens). Each chunk runs causal attention over the native MLX cache to produce correct hidden states, and its K/V are captured into the store. After each chunk, CompressDeferred virtually concatenates the surviving dense tail with the newly captured tokens and compresses every full block that now clears the recency window, in a single SVD batched across all layers and blocks (Algorithm 1 applied in bulk). Streaming compression means peak uncompressed KV during prefill is bounded by roughly one window plus one chunk, not the whole prompt—the difference between fitting a 32k+ prefill in 8 GB and not.

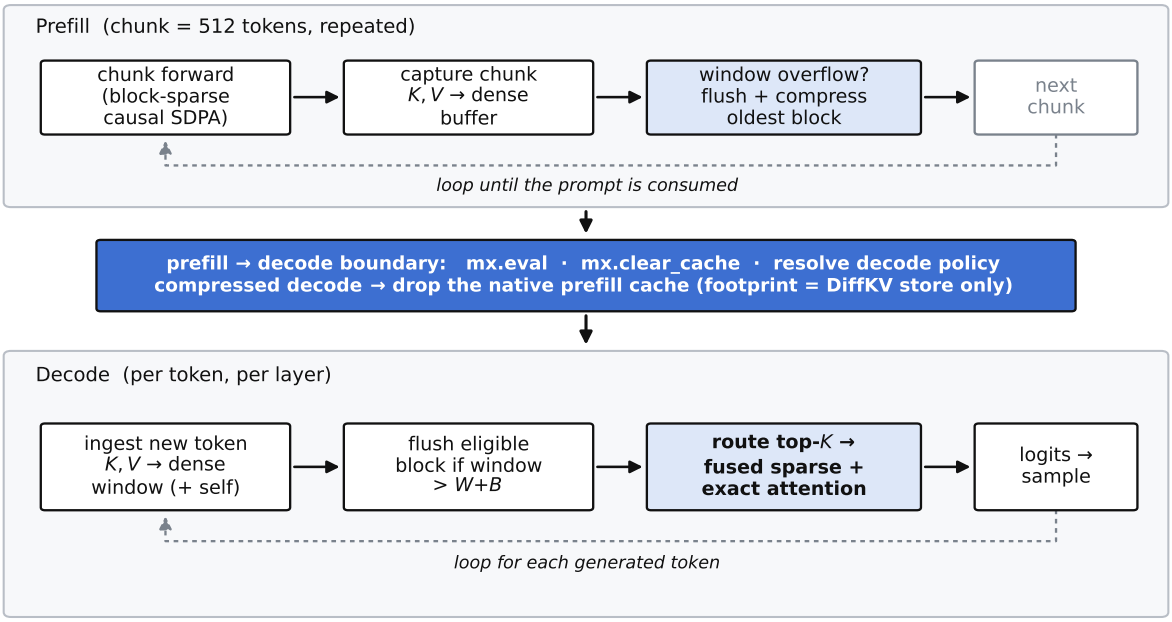

Figure 4 | Cache lifecycle. Prefill loops over chunks (attend, capture, flush-and-compress the oldest block on overflow). At the prefill→decode boundary the runtime evaluates pending work, releases peak activations, and—on the compressed path—drops the native prefill cache. Decode then loops per token: ingest the new token into the window, flush an eligible block, and run the fused routed attention.

Block-sparse prefill. By default, once a chunk is far enough into the prompt that there is prunable history, its attention is computed over a *sparse* key set rather than the full past: leading attention-sink block(s), a set of top-K routed history blocks (selected by a Quest-style min/max bound on the raw block keys), the recency window, and the current chunk itself, combined under one masked attention call. This is training-free block-sparse attention (attention sinks plus block retrieval) on a frozen model; it reduces prefill attention from $O(L^2)$ toward $O(L \cdot K)$ and is the mechanism behind the prefill crossover in §8 (E3). It changes compute, not the stored state: every token's exact K/V is still captured for later compression, so retrieval accuracy is unaffected.

5.3. The prefill→decode boundary

The transition from prefill to decode is where memory is reclaimed (Figure 4). At the first decode step the runtime resolves the decode policy once, flushes pending work with `mx.eval`, and calls `mx.clear_cache` to return the peak GQA-expanded prefill activations to the shared pool. When decode will use the compressed path, it additionally *drops the native prefill KV cache* entirely: decode runs solely on the compressed store plus the dense window, so decode-time memory reflects the DiffKV footprint and not a retained full-context cache.

5.4. Decode ingestion and the sliding window

Each decode step appends the new token's K/V to the dense window and, if the window has grown past $W+B$, flushes and compresses its oldest block into the pool—the same streaming rule

Algorithm 2 Session lifecycle (one request)

init session: pre-allocate dense window + pool of M block slots (per layer)
for each prefill chunk c **do** ▷ L>1
    attend(c) over native cache (block-sparse if enough history); capture K, V
    CompressDeferred: compress full blocks clearing the window (batched SVD)
end for
boundary: resolve decode policy; `mx.eval`; `mx.clear_cache`
if compressed decode **then** drop native prefill KV cache
end if
for each decode step **do** ▷ L=1
    ingest new token K, V into dense window; flush+compress oldest block if overflow
    $o \leftarrow$ FusedRoutedAttention(store, q) ▷ Alg. 3
    sample next token from logits
end for

as prefill. The current token is always in the exact window, so a token attends to itself and its immediate neighbors exactly; only older context is served from the compressed pool. The attention itself is the subject of §6.

6. Fused Routed Decode Attention

Decode is where compression must pay off without materializing the cache. The kernel (Figure 5; `compute_decode_attention_static`, compiled with `mx.compile`) computes, for a single query $q \in \mathbb{R}^{H_q \times d}$, an attention output over the compressed pool and the dense window as two halves merged in log-space.

6.1. Routing: bound the work, not the context

Attending and reconstructing every block would make decode scale with context. Instead a router scores all blocks cheaply and keeps the top K=16. The default router is *exact*: it scores q against each block's anchor key and its most distinctive residual keys, and ranks blocks by the largest true dot product,

$$\rho_b \;=\; \max\Big(q \cdot a_k^{(b)},\; \max_{j \le R} q \cdot R_{K,j}^{(b)} \Big) \cdot \text{scale}. \tag{5}$$

Because the residuals *are* the block's outlier tokens, scoring them gives a tight relevance signal that reliably keeps a needle's block in the top-K even at large block counts—where a min/max box bound (also available, and cheaper, in the style of Quest [13]) becomes too loose. Routing costs $O(\text{nb} \cdot R \cdot d)$ per token and is the dominant decode cost at long context, so the number of residuals *scored* by the router is decoupled from the number *stored* (§7).

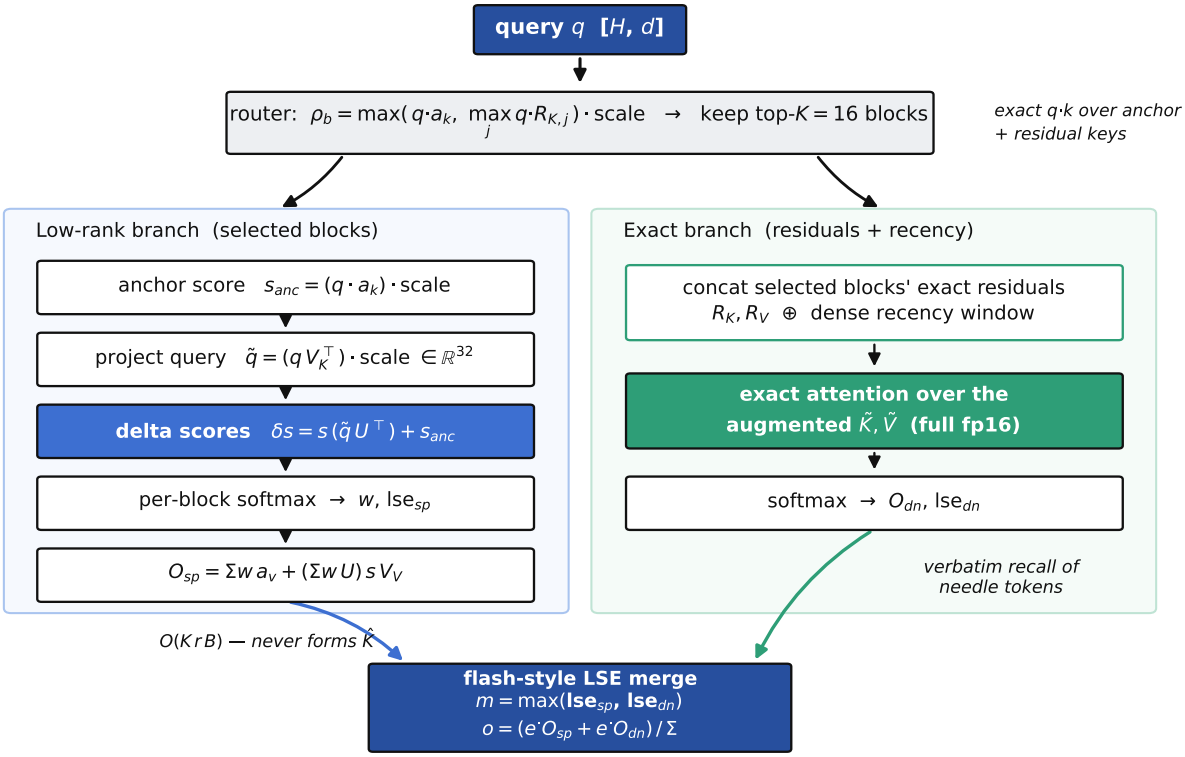

Figure 5 | The fused decode kernel. A cheap exact router keeps the top-K blocks. Their contribution is computed in the rank-r subspace—anchor score plus a projected-query dot with U—without ever forming $\hat{K}$. In parallel, the selected blocks' exact residuals and the dense recency window are attended directly. The two halves are combined by a flash-style log-sum-exp merge, guarded so an empty compressed set contributes exactly zero weight.

6.2. Sparse half: scoring in the low-rank subspace

For each selected block the kernel scores q against every token *without reconstructing* K. Using Eq. (4), the score of token i decomposes into an anchor term and a delta term:

$$q \cdot \hat{K}_i = \underbrace{q \cdot a_k}_{\text{anchor score}} + s \underbrace{\left(q\, V_K^\top \right)}_{\tilde{q} \in \mathbb{R}^r} \cdot U_i. \tag{6}$$

The query is projected once into the rank-r subspace ($\tilde{q} = qV_K^\top$), and all $B-1$ token scores in a block are then a single $\tilde{q}\, U^\top$ product. This is $O(K(H_{\text{kv}} r d + rB))$ per token—linear in the rank and independent of d in the inner loop—rather than the $O(K\, B\, d)$ of decompress-then-score. The value side is symmetric: softmax weights are contracted with U and then with V_V, plus the anchor contribution, giving the sparse output O_{sp} and its log-sum-exp lse_{sp}. Residual positions are masked out of this pool so their exact copies (below) are their sole representation.

6.3. Exact half: residuals and recency

The selected blocks' exact residual keys/values are concatenated with the dense recency window and attended directly in fp16—an ordinary masked attention producing O_{dn} and lse_{dn}. This half

Algorithm 3 Fused routed decode attention for query q (one layer)

Require: query q; pool $\{U, V_K, V_V, a_k, a_v, s, R_K, R_V\}_b$; dense window K_d, V_d

$\rho_b \leftarrow \max(q \cdot a_k^{(b)}, \max_j q \cdot R_{K,j}^{(b)}) \cdot \text{scale}$ ▷ Eq. (5)

$\mathcal{S} \leftarrow$ top-K blocks by ρ_b

for $b \in \mathcal{S}$ **do** ▷ scored in rank space, no $\hat{K}$

  $\tilde{q} \leftarrow q\, V_K^{(b)\top}$; $\text{scores}_i \leftarrow q \cdot a_k^{(b)} + s\, \tilde{q} \cdot U_i^{(b)}$

  mask residual positions to $-\infty$

end for

$O_{\text{sp}}, \text{lse}_{\text{sp}} \leftarrow$ softmax/combine over pooled block scores with V side of Eq. (4)

$O_{\text{dn}}, \text{lse}_{\text{dn}} \leftarrow$ exact attention over $[\, R_{K,\mathcal{S}} \,\|\, K_d \,], [\, R_{V,\mathcal{S}} \,\|\, V_d \,]$

return flash-merge$(O_{\text{sp}}, \text{lse}_{\text{sp}}, O_{\text{dn}}, \text{lse}_{\text{dn}})$ ▷ Eq. (7)

is where a buried passcode is recovered verbatim: its token, being a high-error outlier, was kept exact as a residual, so the query attends to the true key and copies the true value.

6.4. Flash-style merge

The two halves are combined by the standard online-softmax identity used by FlashAttention [2], in a numerically guarded form:

$$m = \max(\text{lse}_{\text{sp}}, \text{lse}_{\text{dn}}), \qquad o = \frac{e^{\text{lse}_{\text{sp}} - m}\, O_{\text{sp}} + e^{\text{lse}_{\text{dn}} - m}\, O_{\text{dn}}}{e^{\text{lse}_{\text{sp}} - m} + e^{\text{lse}_{\text{dn}} - m}}. \tag{7}$$

NaN/inf guards make an empty compressed set contribute exactly zero weight, so the kernel degrades to pure dense-window attention with no special case. We keep the score and combine arithmetic in the precision the kernel was validated at (fp16 products with fp32 log-sum-exp accumulation); recasting the merge to fp32 was measured to perturb long-context retrieval and is avoided (§7). An optional additive bias on the compressed half's log-sum-exp corrects a systematic under-weighting of the low-rank pool for diffuse multi-fact queries; it defaults to a no-op and does not affect the exact single-needle recall reported here.

6.5. Decompress-and-cache

Reconstructing the routed blocks *once* and reusing them across several decode steps trades a little memory for throughput. When enabled (the default in the serving configuration used for our measurements), the kernel materializes the selected blocks' K/V into a contiguous buffer via Eq. (4), overwrites the residual rows with their exact copies, and attends [cached blocks ‖ window] with a single fused attention; it re-routes and re-materializes every N tokens (or whenever the block set changes). This is bit-exact to the per-token kernel and reduces per-token work, at the cost of a buffer bounded by K (not by context). Its effect on throughput is part of the measured decode numbers in §8.

7. Implementation Notes

DiffKV's active runtime is $\approx 3{,}500$ lines of Python over MLX [6], evaluated on Qwen2.5-1.5B [12]. A few implementation choices are load-bearing and worth recording, because they are where cor-

rectness or performance was actually won or lost.

Attention by monkey-patch. Rather than fork `mlx_lm`, the runtime replaces `qwen2.Attention.__call__` at load time with the patched forward of §3, and hands each layer a reference to the shared block manager. This keeps the runtime a thin, model-agnostic overlay on an unmodified upstream model.

Randomized SVD in NumPy. Compression runs the randomized truncated SVD in fp32 in NumPy on the CPU, not in MLX on the GPU. On unified memory this is nearly free to marshal (no host–device copy), and it sidesteps GPU-only limitations in the current MLX linear-algebra path (e.g. `qr`); a 255×512 block factorizes in ≈ 2 ms. The seed is fixed so that compression—and therefore decode output and needle recovery—is deterministic run to run; an unseeded range finder made recall vary between identical runs.

Decode precision. The decode score/merge arithmetic uses fp16 products with fp32 log-sum-exp accumulation. A well-intentioned change that recast the merge operands to fp32 for "overflow safety" was measured to shift the combine by roughly fp16-epsilon and *regress* long-context (16k/32k) needle recall into a repetition loop, because at those lengths the retrieval margin sits on a knife edge. The fp32-log-sum-exp / fp16-combine split is the validated configuration and is enforced by a parity oracle in the test suite.

Router/attention decoupling. The number of residuals the router *scores* (Eq. (5)) is a separate knob from the number a block *stores*. The router cost is $O(\text{nb} \cdot R_{\text{route}} \cdot d)$ per token and dominates long-context decode, so R_{route} defaults below the storage budget R; raising R for prose fidelity therefore does not automatically inflate router cost.

Configuration. Behavior is controlled by environment flags with safe defaults, so new paths ship disabled until validated. The defaults used throughout this report are: compressed sparse decode on; decompress-and-cache on; block-sparse prefill on; rank r=32; R=128; top-K = 16; residual-key router; SVD seed 1234. The dense baseline is `mlx_lm` with a full KV cache on the same int4 weights.

Gated subsystems. A factual-store / semantic-retrieval module and a speculative-decode draft path exist in the codebase but are disabled by default (they are no-ops in the configuration measured here) and are not part of the evaluated system. We mention them only to be exact about what is and is not exercised.

Testing. A kernel-parity oracle checks the fused decode kernel against a reference implementation on seeded sessions; the needle and relational-binding harnesses guard recall. The measurement scripts, prompts, and the exact commands are in the reproducibility appendix (Appendix A).

Table 2 | Main results: DiffKV vs. a dense full-KV baseline on identical int4 weights (Apple M3, 8.6 GB; greedy, 128 decode tokens). All cells measured.

Metric	4k	8k	16k	32k	64k
DiffKV active runtime (compressed sparse decode)					
Prefill (s)	6.6	13.6	28.2	58.5	928.0
Decode (tok/s)	19.9	18.4	18.7	17.0	8.6
MLX peak (GB)	1.74	1.89	2.36	3.12	4.63
Needle recovered	✓	✓	✓	✓	✓
Dense baseline (mlx_lm, full KV, same weights)					
Prefill (s)	5.1	11.8	27.8	77.9	*OOM*
Decode (tok/s)	65.7	55.3	47.0	35.7	*OOM*
MLX peak (GB)	1.68	1.79	2.03	2.45	*OOM*
Needle recovered	✓	✓	✓	✓	*OOM*

8. Evaluation

8.1. Setup

All numbers are measured on one host: an Apple M3 with 8.6 GB of unified memory, running Qwen2.5-1.5B at int4 via `mlx_lm`. We compare two engines on the *same* quantized weights: **DiffKV** (the active runtime in its default serving configuration—compressed sparse decode, decompress-and-cache, block-sparse prefill, r=32, R=128, K=16) and **dense** (`mlx_lm` with a full KV cache). Running both on identical weights makes the comparison a clean ablation of the cache representation alone.

The workload is a needle-in-a-haystack (NIAH) prompt [7]: a unique passcode is buried at ∼50% depth in a technical-prose haystack, and the request asks the model to state the passcode and then write a long explanation. The same deterministic prompt is used for every context length and both engines. We report prefill time, decode throughput (greedy, a fixed 128 tokens with EOS ignored so throughput is comparable), MLX allocator peak, and whether the exact passcode appears in the output. Contexts span 4k to 64k. Memory is the MLX allocator peak (`mx.get_peak_memory`), the allocator-true metric; the analytic KV-state footprint is computed from the store layout (Table 1). Table 2 is the headline; the per-run detail is in Appendix B.

8.2. E1 — KV-state footprint

Figure 6 plots the analytic KV state against context. A dense cache grows linearly and without bound; DiffKV's store grows more slowly and is capped by the fixed 256-block pool. At the default R=128 the per-block ratio is 1.44×; the R=64 memory preset reaches 2.25× and keeps the whole store under the pool cap even at 64k. The point is not a large constant factor—the residual pool deliberately spends bytes to preserve recall—but that the state is *bounded*: past the pool's capacity the cache stops being the term that grows, which is what decides whether long context fits on a memory-constrained device.

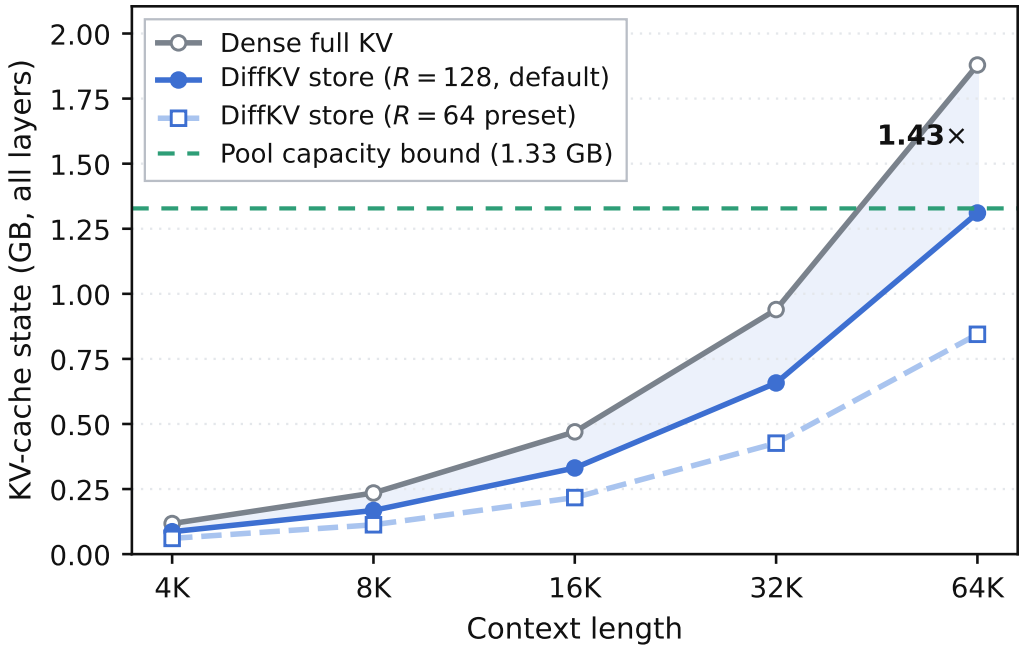

Figure 6 | Analytic KV-state footprint vs. context (whole model). The dense cache grows linearly; the DiffKV store grows sub-linearly and is bounded by the pre-allocated 256-block pool. Both residual presets are shown; the R=64 preset stays under the pool cap through 64k.

8.3. E2 — Exact recall and reach

Across every context from 4k to 64k, DiffKV recovers the buried passcode *exactly* (Table 2, "Needle recovered"); the dense baseline also recovers it wherever it runs. This is the load-bearing correctness result for DiffKV: compression to a low-rank basis would ordinarily corrupt such a token, and the residual mechanism (§4) is what preserves it. Crucially, at 64k the dense full-KV cache *exhausts the host's* 8.6 *GB and does not complete* (a reproducible out-of-memory during prefill), while DiffKV finishes with the passcode intact at a 4.63 GB allocator peak. This is the reach the bounded store buys: a context the dense cache cannot hold at all.

8.4. E3 — Prefill scaling and the crossover

DiffKV's prefill does less attention work than dense (block-sparse, $O(L \cdot K)$ vs. $O(L^2)$), so its cost grows sub-quadratically while dense grows quadratically (Figure 7a). At short context the fixed cost of streaming SVD compression makes DiffKV slightly slower (6.6 s vs. 5.1 s at 4k); the two are essentially matched by 16k (28.2 s vs. 27.8 s), and DiffKV pulls ahead by 32k, prefilling in 58.5 s versus dense's 77.9 s (1.33×). The crossover is where the quadratic term in dense attention overtakes the fixed compression overhead, and it moves earlier as either the model or the context grows. At 64k the comparison ends differently: DiffKV completes prefill in 928 s (heavily memory-bound on this 8.6 GB host, so reported as a reach point rather than a clean timing), while the dense baseline does not complete at all—it exhausts memory during prefill (E2).

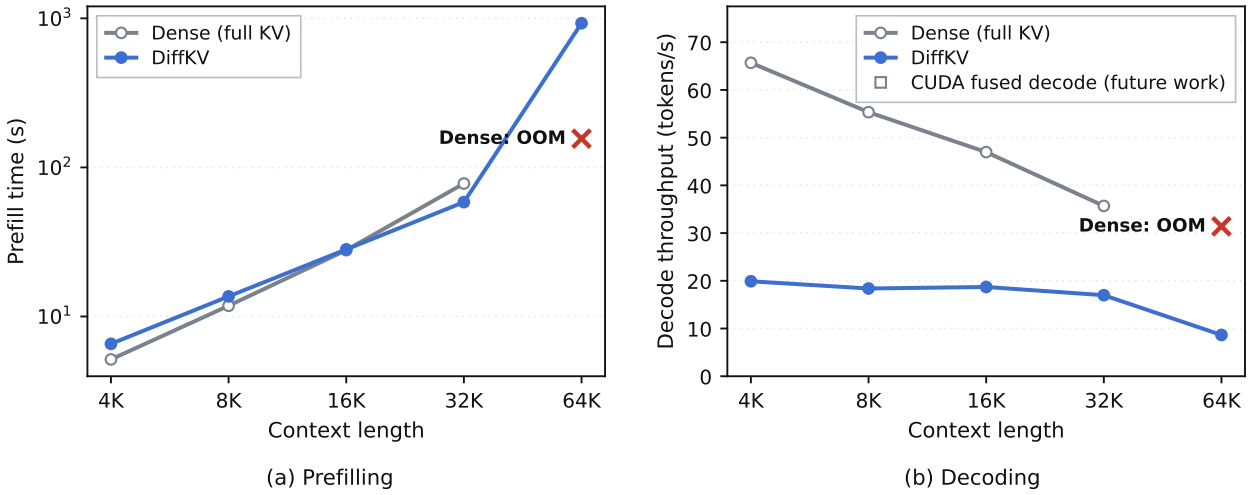

Figure 7 | Prefill and decode of DIFFKV versus the dense full-KV baseline on identical int4 weights (Apple M3, 8.6 GB; greedy, 128 decode tokens). (a) Block-sparse prefill grows sub-quadratically and overtakes the quadratic dense baseline by 32K; at 64K the dense baseline runs out of memory during prefill, while DIFFKV completes. (b) DIFFKV decode is nearly flat in context but slower than dense wherever the dense cache still fits.

8.5. E4 — Decode throughput (the cost)

We state the cost of compression plainly. DIFFKV's decode reconstructs and merges a sparse representation every token, which a dense cache does not; where the dense cache still fits in memory—which, for this 1.5B model on 8.6 GB, it does through 64k—dense decodes faster (Figure 7b, Table 2). With decompress-and-cache enabled, DIFFKV decode is nearly *flat* across context (19.9 → 17.0 tok/s from 4k to 32k—the top-K routing bounds the per-token work independent of length), while dense declines steadily (65.7 → 35.7 tok/s as its cache grows). The gap therefore *narrows* as context grows—from over 3× at 4k to 2.1× at 32k—but does not close within the range where the dense cache still fits. DIFFKV's value proposition is bounded memory, exact recall, and prefill scaling—not decode speed—and a faster decode path (a fused kernel; a CUDA/Triton port, §10) is the clearest avenue to narrow this gap further.

8.6. E5 — Residual budget: the accuracy/memory/speed dial

R is the central design knob. Table 3 sweeps it at 16k under forced compressed decode (rank-32 store). As R falls, the compression ratio rises steeply—from 1.40× at R=128 to 3.80× at R=8—while decode stays fast (fewer exact tokens to attend). Single-needle recall is preserved across this whole range: the passcode's outlier token, being a high-reconstruction-error row, is captured as a residual even at a small budget, so a compact store already suffices for verbatim retrieval at the default rank. The budget's remaining role is prose-fact fidelity (multi-fact recall of names/places, where the low-rank part alone confabulates and the larger default budget is needed) and robustness margin for harder or deeper needles. The trade-off is thus a clean memory–speed dial with a wide recall-preserving operating range. (We note a degenerate boundary: at R=0 there are no residual keys for the exact-key router to score, so that configuration is not a valid operating point—the method is defined for $R \geq 1$.)

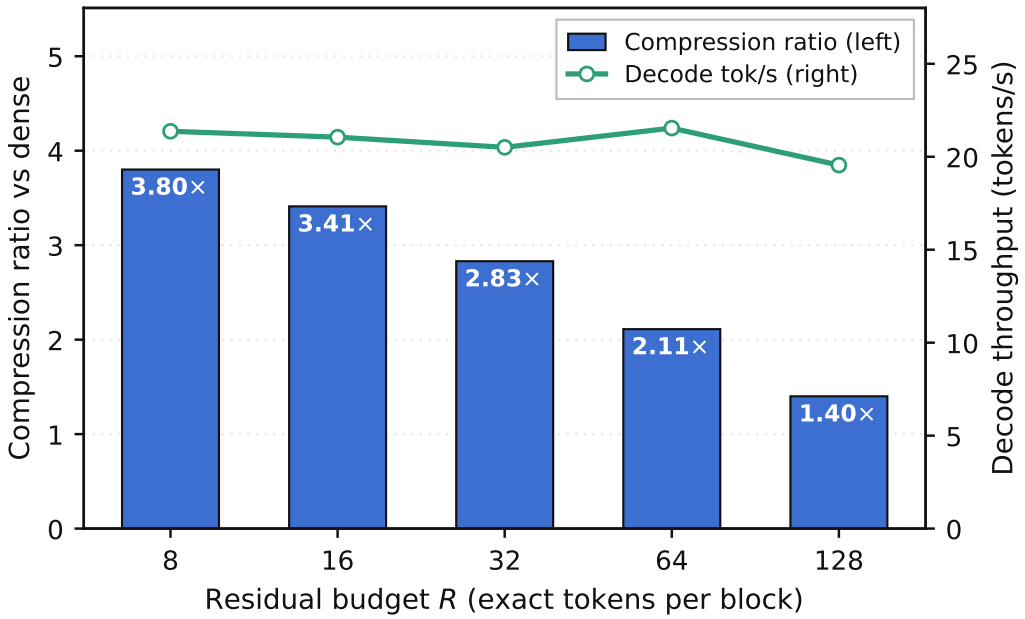

Figure 8 | Residual-budget sweep at 16K (forced compressed decode, rank-32 store). Bars: compression ratio versus dense (left axis); line: decode throughput (right axis). The needle is recovered exactly at every budget in this range (Table 3), so R acts as a clean memory–speed dial, with the larger default retained for prose-fact fidelity.

8.7. E6 — Where the decode cost comes from

To localize the decode cost, we decode over the *same* compressed store two ways: the real compressed sparse kernel, and an "exact" full-precision decode over the reconstructed store (Table 4, Figure 9). Both recover the needle at every context, so the gap between them is purely the overhead of scoring and merging the sparse representation rather than attending dense tensors—it isolates the reconstruction-and-merge cost from the retrieval quality, and shows the correctness of the sparse path is not what limits its speed.

9. Analysis and Discussion

9.1. What DiffKV buys, and what it costs

The evaluation supports a precise, bounded claim. DiffKV converts the KV cache from an *unbounded, growing* liability into a *bounded, fixed-pool* one, at 1.44×–2.25× per-block compression, while preserving exact recall of distinctive tokens and making long-context prefill sub-quadratic. It costs per-token decode throughput, because the sparse representation must be scored and merged every step where a dense cache would simply attend. Through 32k, where the dense cache still fits on our host, DiffKV is the slower decode and dense is the right choice; at 64k the dense cache no longer fits at all (it OOMs during prefill) and DiffKV is the only one that runs. That boundary is the whole point: DiffKV's regime is where the cache would otherwise not fit—larger models, longer contexts, tighter memory, or more concurrent sessions—and there the alternative to a slower decode is no decode at all.

Table 3 | Residual-budget sweep at 16K (rank-32 store, forced compressed decode). Store size and compression ratio are measured; the needle is recovered exactly at every budget shown.

R	Needle	Decode (tok/s)	Store (GB)	Ratio vs dense
8	✓	21.4	0.124	3.80×
16	✓	21.1	0.139	3.41×
32	✓	20.5	0.167	2.83×
64	✓	21.5	0.224	2.11×
128	✓	19.6	0.338	1.40×

Table 4 | Compressed-vs-exact decode over the same DiffKV store (mechanism ablation).

Decode over the same DiffKV store	4k	8k	16k	32k
Compressed sparse decode (tok/s)	20.0	18.4	18.8	16.4
Exact decode, upper bound (tok/s)	36.7	33.6	30.3	22.3
Compressed — needle	✓	✓	✓	✓
Exact — needle	✓	✓	✓	✓

9.2. Why decode is flat but slow

Two facts from §8 explain the decode shape. It is *flat* in context because top-K routing bounds the expensive work to a constant number of blocks and the decompress-and-cache path reconstructs them once per interval rather than per token; adding context adds pool slots but not per-token work. It is *slower than dense* because that constant of work—projecting the query into the rank subspace, contracting with U and V_V, attending the residual pool, and merging two log-sum-exp halves—is simply more per-token arithmetic than a single fused attention over a cache that still fits. The E6 ablation confirms the gap is reconstruction-and-merge overhead, not a correctness compromise. This is why the highest-leverage optimization is a single fused decode kernel (or a GPU-native CUDA/Triton one), not a change to the representation.

9.3. The memory story, stated carefully

We are deliberately careful about memory because it is the easiest claim to overstate. The *analytic KV state* is unambiguously smaller and bounded (E1). The *measured allocator peak*, through 32k, is close between the two engines, and DiffKV's can even exceed dense's, because the peak there is set during prefill and by the pre-allocated pool rather than by steady-state decode; on a 1.5B model at those lengths, full KV is simply not yet large enough for the compression to dominate the peak. The claim sharpens at the boundary: at 64k the dense cache *does* become the binding constraint and OOMs, while DiffKV's bounded store completes. So the honest memory story has two parts—below the boundary, compression buys a smaller *growth rate* but not a smaller measured

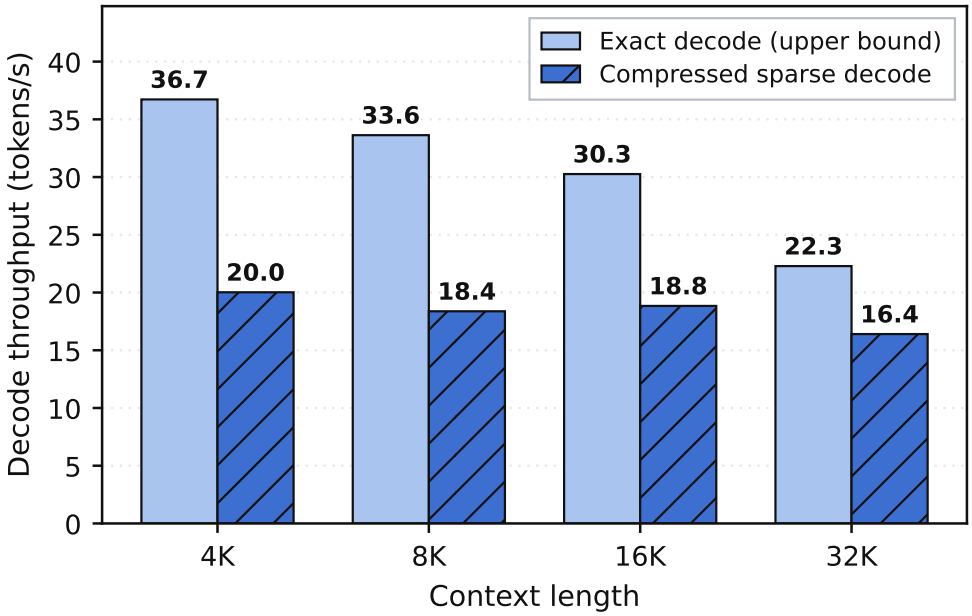

Figure 9 | Decoding the same store two ways: compressed sparse vs. exact upper bound. Both recover the needle; the difference isolates the per-token reconstruction/merge overhead.

peak; at and beyond the boundary, the bound is what makes the context runnable at all. The peak advantage is a reach advantage.

9.4. When does DiffKV win?

Synthesizing the results: DiffKV wins when (i) the KV cache, not the weights, is the memory bottleneck—bigger models, longer contexts, many concurrent sessions; (ii) prefill dominates the request (long prompt, short answer), where sub-quadratic block-sparse prefill is a direct wall-clock win; and (iii) exact recall of specific tokens matters, where the residual mechanism preserves fidelity that quantization or pure low-rank would lose. It loses on pure decode throughput whenever a dense cache fits. Stating both halves is the point: the design is a memory-and-prefill instrument with a decode cost, not a free lunch.

9.5. Relationship to sparse-attention methods

DiffKV's decode is, in effect, a training-free instance of retrieve-then-attend sparse attention: the low-rank block reconstruction plays the role of a coarse compressed representation, the exact residuals and top-K routing play the role of fine token selection (cf. query-aware block sparsity [13] and heavy-hitter selection [16]), and the recency window is the local branch [15], combined by an exact log-sum-exp merge [2]. The difference from methods that require a trained importance predictor is that DiffKV's router scores exact residual keys directly, so it needs no training and no tuning to head count or content—the residuals are whatever a given block's distinctive tokens happen to be.

10. Limitations and Future Work

We list the limitations plainly; several are direct consequences of the design choices above.

Decode throughput. The headline cost. DiffKV decode is slower than a dense cache that fits (§8, E4), and the E6 ablation localizes the gap to per-token reconstruction-and-merge overhead. The clearest remedy is a single fused decode kernel that does the routing, low-rank scoring, residual attention, and merge in one launch, rather than a sequence of MLX ops.

CUDA / Triton path (reserved, not evaluated). A CUDA / Triton decode kernel exists in the repository, but the evaluation host has no NVIDIA GPU, so we report *no* CUDA numbers and make *no* CUDA claims. The decode design ports cleanly—reconstruct routed blocks into a buffer (a matmul), attend with a flash kernel, cache across a re-route interval—so we expect the gap in E4 to narrow on a GPU-native path, but that is a hypothesis to be measured, not a result. The figures and tables reserve space (a series slot in Figure 7, a column convention in the results tables) so CUDA data can be inserted without restructuring the paper.

Single model, single host. Every number here is Qwen2.5-1.5B (int4) on one Apple M3. The residual budget and router settings are tuned for this model; larger models, different architectures, and non-Apple hardware are unvalidated. Two normalizations (the V-to-K RMS rescale, the seeded rank) are model-sensitive and should be re-checked per model.

Evaluation breadth. Correctness here is exact single-needle recall plus fluent long-form generation. This is a strong, unambiguous signal for the fidelity claim, but it is not a substitute for standard long-context suites (RULER, LongBench), multi-needle and multi-hop recall, or comparisons against other cache-compression baselines (quantization, eviction, other low-rank methods). Diffuse multi-fact synthesis, where the top-K router must cover several regions at once, is a known weaker case that the single-needle metric does not probe; an optional log-sum-exp bias mitigates it but is not a full solution. Broadening the evaluation is the most important next step for external credibility.

Memory-bound long context on an 8 GB host. At 64k, both engines are pushed to the machine's limit: DiffKV's prefill completes but is heavily memory-bound (so its 64k prefill *time* is a reach indicator, not a clean measurement), and the dense baseline OOMs outright. A machine with more memory headroom would give cleaner absolute long-context timings and would move the dense OOM boundary further out; the robust claims are the sub-quadratic prefill scaling and the bounded KV state, not the specific 64k prefill seconds.

Bounded pool. The fixed 256-block pool caps compressed capacity at $65{,}536$ tokens. Beyond that, the oldest blocks must be dropped or the pool enlarged (trading memory). A capacity policy for contexts past the pool is future work.

11. Conclusion

DiffKV is a KV-cache compression runtime that separates a block's smooth, shared structure—captured by an anchor and a rank-r joint $K \mid V$ delta—from its distinctive outlier tokens, kept as exact residuals selected by measured reconstruction error. A fused, routed decode kernel scores queries in the low-rank subspace without ever decompressing the cache, attends the residuals and a recency window exactly, and merges the two in log-space; a training-free block-sparse prefill makes long-context prefill sub-quadratic; and a fixed block pool bounds the memory. Implemented entirely in MLX and measured on Qwen2.5-1.5B (int4) on an 8.6 GB Apple M3, DiffKV holds a bounded KV state $1.44\times$–$2.25\times$ smaller per block than a dense cache, recovers a buried passcode exactly from 4k to 64k, reaches 64k where the dense full-KV cache runs out of memory, and overtakes dense on long-context prefill—at a measured cost in decode throughput, in the regime where a dense cache still fits, that we report in full and localize to reconstruction-and-merge overhead. The residual budget is an explicit dial between memory, accuracy, and speed. The most promising next steps are a fused (and GPU-native) decode kernel to narrow the throughput gap, and a broader evaluation across models, hardware, and standard long-context suites. We have been careful to claim only what the measurements support: DiffKV is a memory-and-prefill instrument for long context on commodity unified-memory hardware, not a universally faster one.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[2] Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[3] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint, 2024. Introduces Multi-head Latent Attention (MLA), a low-rank latent KV cache.

[4] Harry Dong, Xinyu Yang, Zhenyu Zhang, Zhangyang Wang, Yuejie Chi, and Beidi Chen. Get more with LESS: Synthesizing recurrence with KV cache compression for efficient LLM inference. arXiv preprint, 2024.

[5] Nathan Halko, Per-Gunnar Martinsson, and Joel A Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

[6] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. https://github.com/ml-explore/mlx, 2023.

[7] Greg Kamradt. Needle in a haystack – pressure testing LLMs. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large

language model serving with PagedAttention. In *Symposium on Operating Systems Principles (SOSP)*, 2023.

[9] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. arXiv preprint, 2024.

[10] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[11] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. arXiv preprint, 2024.

[12] Qwen Team. Qwen2.5 technical report. arXiv preprint, 2024.

[13] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2024.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[15] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

[16] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A. Reproducibility

All results were produced on a single machine, one engine per isolated subprocess, with every number measured (failed cells are reported as failures, never filled with estimates).

Hardware. Apple M3, 8.6 GB unified memory, macOS (Darwin 25.x). The device is shared with the OS and other applications, so absolute memory thresholds and long-context prefill times shift with machine load; the scaling slopes, engine ordering, and exact-recall results are the stable outcomes.

Software. Python in `diffkv_venv`, MLX $0.31.2$, `mlx_lm`, NumPy $2.x$, PyTorch (used only for tensor plumbing in the wrapper, not for compute), `psutil`. Model: `mlx-community/Qwen2.5-1.5B-Instruct-4bit` (group size 64, 4-bit), 28 layers, 12 query heads, 2 KV heads, head dim 128, trained context $32{,}768$, RoPE $\theta = 10^6$.

Configuration (as measured). preset `mid`, `serving_mode=balanced`, `rank=32` (the "mid" preset default; only the "low" preset reduces it to 16), `block_size=256`, `recency_window=512` (dense buffer 768), `max_blocks=256`, `max_residual=128`, top-K=16 routing with the `residual` router, `DIFFKV_SVD_SEED=1234` (deterministic). The DiffKV engine runs in the active CLI's serving configuration with the sparse path forced on at every context: `DIFFKV_COMPRESSED_DECODE=1` (so DiffKV's compressed decode is what is measured at all lengths, not the adaptive dense-below-threshold policy), `DIFFKV_DECODE_CACHE=1` (decompress-and-cache; bit-exact), `DIFFKV_SPARSE_PREFILL=1` (block-sparse prefill), `DIFFKV_SPARSE_BIAS=auto` (adaptive merge bias, as the CLI ships). The dense baseline is `mlx_lm` with a full KV cache on the same int4 weights. The residual sweep (E5) forces `DIFFKV_COMPRESSED_DECODE=1` and varies `DIFFKV_MAX_RESIDUAL`; the decode-mode ablation (E6) toggles `DIFFKV_COMPRESSED_DECODE` between 1 (compressed) and 0 (exact upper bound) over the same store.

Protocol. A needle-in-a-haystack chat prompt is built once per context length from the Qwen2.5 tokenizer and fed verbatim (raw mode, no double chat-templating) to every configuration. The needle is a planted passcode (`OMEGA-7741-DELTA`); "needle recovered" means the generation reproduced it. Engines are warmed once (a 1-token forward to compile kernels) before timing. Prefill is timed with `perf_counter` around the chunked forward after `mx.eval`; decode is fixed-length greedy (128 tokens, EOS ignored) so throughput is directly comparable. Memory is read from MLX's allocator (`get_peak_memory`). Each `(engine, context)` cell runs in its own subprocess so memory is attributable and an out-of-memory event kills only that cell. The full primary sweep is a single back-to-back batch, so DiffKV and dense are measured under the same machine conditions; the volatile long-context dense cells were re-measured in isolation to confirm their regime.

Commands.

```
# fresh primary sweep (active vs dense, 4k--32k), isolated per cell
CTXS="4096 8192 16384 32768" bash paper/scripts/measure_sweep.sh

# 64k reach point + long-context dense re-check
bash paper/scripts/measure_tail.sh

# decode-mode ablation (compressed vs exact, same store)
DIFFKV_DECODE_CACHE=1 diffkv_venv/bin/python3 paper/scripts/measure_active.py \
--ctx 4096 8192 16384 32768 --modes compressed exact --gen 128 \
--out paper/generated/active_modes_fresh.json

# residual-budget sweep at 16k
diffkv_venv/bin/python3 paper/scripts/measure_residual_sweep.py \
--ctx 16384 --residuals 0 8 16 32 64 128 \
--out paper/generated/residual_sweep_fresh.json

# facts, figures, and tables from the measured JSON (no hand-typed numbers)
diffkv_venv/bin/python3 paper/scripts/make_facts.py
diffkv_venv/bin/python3 paper/scripts/make_diagrams.py
diffkv_venv/bin/python3 paper/scripts/make_graphs.py
diffkv_venv/bin/python3 paper/scripts/make_tables.py
```

Artifacts and provenance. Primary measured JSON: `benchmarks/results/fresh_{active,dense}_{ctx}.json`. Ablations: `paper/generated/active_modes_fresh.json` (decode modes) and `paper/generated/residual_sweep_fresh.json` (residual budget). Every figure and table is regenerated from these by the scripts above; the body text's numbers are LaTeX macros emitted by

`make_facts.py`, so the prose cannot drift from the data. The dataset selection, the decode-path attribution that motivated this measurement design, and the corrected compression-ratio accounting are documented in `paper/notes/AUDIT_2026-07-07.md` and the consistency report.

B. Full Measured Results

Table 5 lists every cell of the fresh primary sweep (`paper/scripts/measure_sweep.sh` and `measure_tail.sh`) for both engines, exactly as recorded by the isolated workers. *MLX peak* is the allocator peak (`get_peak_memory`) inside the worker process; *Prompt tok* is the tokenizer count for the built NIAH prompt at that context. Empty cells are measurements that do not apply; failed cells would be stated as failures in the text, never given an invented value.

Table 5 | Per-run detail for the primary sweep: DiffKV (active) and the dense baseline on identical int4 weights. All values measured.

Engine	Ctx	Prompt tok	Prefill (s)	Decode (tok/s)	MLX peak (GB)	Needle
active	4k	4096	6.6	19.9	1.74	✓
active	8k	8192	13.6	18.4	1.89	✓
active	16k	16490	28.2	18.7	2.36	✓
active	32k	32865	58.5	17.0	3.12	✓
active	64k	65615	928.0	8.6	4.63	✓
dense	4k	4096	5.1	65.7	1.68	✓
dense	8k	8192	11.8	55.3	1.79	✓
dense	16k	16490	27.8	47.0	2.03	✓
dense	32k	32865	77.9	35.7	2.45	✓

The configuration table (Table 6) records the model and runtime parameters as read from the code, and Table 1 in §4 derives the per-block byte budget from those same dimensions.

Table 6 | Model and DiffKV configuration, as measured/loaded by the runtime.

Parameter	Value
Model	Qwen2.5-1.5B-Instruct (int4)
Transformer layers L	28
Query / KV heads (GQA)	12 / 2
Head dimension d	128
Block size B	256 tokens
SVD rank r	32 (adaptive $\leq r$, 99.9% energy)
Recency window W (+block)	512 (+256 = 768 exact)
Residual budget R	128 (default) / 64 (memory preset)
Block pool M	256 blocks ($\leq$ 65 536 tokens)
Top-K routed blocks	16 (residual-key router)
Store dtype	fp16
SVD seed	1234 (deterministic)

C. Notation

Symbol	Meaning
L	transformer layers (28)
H	query heads (12)
H_{kv}	key/value heads (2)
$G = H/H_{\text{kv}}$	GQA group size (6)
d	head dimension (128)
N	sequence length (tokens)
B	block size (256)
$S = B - 1$	compressible tokens per block (255)
r	SVD target rank (32, "mid" preset; adaptive in $[4, r]$)
R	exact residual budget per block (128 default; 64 memory preset)
K	top-K blocks routed at decode (16)
W	dense recency window (512; buffer $W+B = 768$)
M	compressed-pool capacity (256 blocks)
a_k, a_v	anchor key / value (token 0 of a block)
U	per-token coefficients $\mathbb{R}^{S\times r}$ (shared K/V)
V_K, V_V	key / value bases $\mathbb{R}^{H_{\text{kv}}\times r\times d}$
R_K, R_V	exact residual keys / values $\mathbb{R}^{R\times H_{\text{kv}}\times d}$
$k^{\min}, k^{\max}$	per-block element-wise key min / max (router)
ρ_b	router relevance of block b (Eq. 5)
s	SVD normalization scale (fp32)
q, σ	query, attention scale
$\ell^{\text{sp}}, \ell^{\text{dn}}$	log-sum-exp of low-rank / exact branch